

User Registration Form with Client-Side Validation

AIM

To design and implement a User Registration Form using HTML, CSS, and JavaScript that performs client-side validation of user inputs — specifically email address and phone number — using the Validator.js library, and provides real-time feedback to the user upon form submission.

PROCEDURE

Step 1: Create the HTML Structure (indexx.html)

1. Create an HTML file with a form container div having a heading "User Registration Form".
2. Add an input group for Email with a text input field (id="email") and an error message placeholder (id="emailError").
3. Add an input group for Phone Number with a text input field (id="phone") and an error message placeholder (id="phoneError").
4. Include a Submit button and a success message paragraph (id="successMessage").
5. Link the Validator.js CDN and the external script file (scriptt.js) before the closing `</body>` tag.

Step 2: Write the JavaScript Logic (scriptt.js)

1. Attach a submit event listener to the form element and call `e.preventDefault()` to stop default browser behavior.
2. Read and trim the values from the email and phone input fields.
3. Clear all previous error and success messages at the start of each submission.
4. Use `validator.isEmail(email)` to validate email; if invalid, display error message and set `valid = false`.
5. Use `validator.isMobilePhone(phone, 'any')` to validate phone; if invalid, display error message and set `valid = false`.
6. If all validations pass, display the success message "Form submitted successfully!".

Step 3: Apply CSS Styling (stylee.css)

1. Style the body using flexbox, centered content, steel-blue background (#4682b4), and full viewport height.
2. Style `.form-container` with a white background, 350px width, padding, box-shadow, and border-radius.
3. Style inputs to be 100% width with padding and rounded border.

4. Style `.error` class in red for validation error messages below inputs.
5. Style the submit button as full-width green with white text and darker green on hover.
6. Style `#successMessage` as centered, bold, green text below the form.

Step 4: Run the Application

1. Open the terminal and run: `start index.html` to launch the form in a browser.
2. Test invalid inputs — leave fields empty or enter incorrect formats and click Submit to verify error messages appear.
3. Test valid inputs — enter a properly formatted email and a valid phone number to confirm the success message.

SCREENSHOTS

The following screenshots illustrate the code and working output of the experiment:

Source Code

```
index.html X index.html TODO.md JS script.js # style.css
index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>User Registration Form</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10
11 <div class="form-container">
12   <h2>User Registration Form</h2>
13
14   <form id="myForm">
15     <div class="input-group">
16       <label for="email">Email:</label>
17       <input type="text" id="email" placeholder="Enter email">
18       <small class="error" id="emailError"></small>
19     </div>
20
21     <div class="input-group">
22       <label for="phone">Phone Number:</label>
23       <input type="text" id="phone" placeholder="Enter phone number">
24       <small class="error" id="phoneError"></small>
25     </div>
26
27     <button type="submit">Submit</button>
28   </form>
29
30   <p id="successMessage"></p>
31 </div>
32
33 <!-- Validator.js CDN -->
34 <script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>
35 <script src="script.js"></script>
36
37 </body>
38 </html>

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Himesh Niranjhan A\Desktop\UID 9 TH> start index.html
PS C:\Users\Himesh Niranjhan A\Desktop\UID 9 TH> 
```

Fig 1: HTML Structure — index.html

```

JS script.js > ...
1  document.getElementById("myForm").addEventListener("submit", function(e) {
2      e.preventDefault();
3
4      let email = document.getElementById("email").value.trim();
5      let phone = document.getElementById("phone").value.trim();
6
7      let emailError = document.getElementById("emailError");
8      let phoneError = document.getElementById("phoneError");
9      let successMessage = document.getElementById("successMessage");
10
11     emailError.textContent = "";
12     phoneError.textContent = "";
13     successMessage.textContent = "";
14
15     let valid = true;
16
17     // Validate Email
18     if (!validator.isEmail(email)) {
19         emailError.textContent = "Please enter a valid email address";
20         valid = false;
21     }
22
23     // Validate Phone Number
24     if (!validator.isMobilePhone(phone, 'any')) {
25         phoneError.textContent = "Please enter a valid phone number";
26         valid = false;
27     }
28
29     // Success
30     if (valid) {
31         successMessage.textContent = "Form submitted successfully!";
32     }
33 });

```

Fig 2: JavaScript Validation Logic — script.js

```
# styleee.css > #successMessage
1 body {
2   font-family: Arial, sans-serif;
3   background: #4682b4;
4   margin: 0;
5   padding: 0;
6
7   display: flex;
8   justify-content: center;
9   align-items: center;
10  height: 100vh;
11 }
12
13 .form-container {
14   width: 350px;
15   padding: 25px;
16   background: white;
17   box-shadow: 0 0 10px gray;
18   border-radius: 8px;
19 }
20
21 h2 {
22   text-align: center;
23   color: #333;
24   margin-bottom: 20px;
25 }
26
27 .input-group {
28   margin-bottom: 15px;
29 }
30
31 label {
32   display: block;
33   margin-bottom: 5px;
34   color: #333;
35 }
36
37 input {
38   width: 100%;
39   padding: 10px;
40   box-sizing: border-box;
41   border: 1px solid #ccc;
42   border-radius: 4px;
43 }
44
45 .error {
46   color: red;
47   font-size: 13px;
48 }
49
50 button {
51   width: 100%;
52   padding: 10px;
53   background: green;
54   color: white;
55   border: none;
56   cursor: pointer;
57   border-radius: 4px;
58   font-size: 16px;
59 }
60
61 button:hover {
62   background: darkgreen;
63 }
64
65 #successMessage {
66   text-align: center;
67   color: green;
68   margin-top: 15px;
69   font-weight: bold;
70 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Himesh Niranjhan A\Desktop\UID 9 TH> start index.html

PS C:\Users\Himesh Niranjhan A\Desktop\UID 9 TH>

Fig 3: CSS Styling — styleee.css

Program Output

User Registration Form

Email:

Please enter a valid email address

Phone Number:

Please enter a valid phone number

Fig 4: Output — Validation Errors (invalid inputs)

User Registration Form

Email:

Phone Number:

Form submitted successfully!

Fig 5: Output — Success Message (valid inputs)

RESULT

The User Registration Form was successfully implemented and executed. The following outcomes were observed:

- When the form was submitted with empty or invalid email input, the error message "Please enter a valid email address" was correctly displayed in red below the Email field.
- When an invalid or empty phone number was entered, the error message "Please enter a valid phone number" appeared in red below the Phone Number field.
- When valid inputs were provided (e.g., email: himeshniranjhan.a.2024.cse@rajalakshmi.edu.in and phone: 9444103226), the form displayed the success message "Form submitted successfully!" in bold green text.
- The Validator.js library effectively handled both email format validation (isEmail) and international phone number validation (isMobilePhone with 'any' locale).
- The CSS styling rendered a clean, card-style form with a blue background, proper error styling, and a responsive submit button with hover effect.

Thus, the experiment was completed successfully, demonstrating client-side form validation using HTML, CSS, JavaScript, and the Validator.js library.